

25 Years of Program Analysis

Zardus



Program Analysis

Specification

What should hold about the program?

Goal

What do we want to achieve regarding the specification?

Technique

How will we achieve the goal?

Dawn of Computing



1830s

1842



Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data.													Working Variables.				Result Variables.			
						$1V_1$	$1V_2$	$1V_3$	$0V_4$	$0V_5$	$0V_6$	$0V_7$	$0V_8$	$0V_9$	$0V_{10}$	$0V_{11}$	$0V_{12}$	$0V_{13}$	$1V_{21}$ in a decimal fraction.	$1V_{22}$ in a decimal fraction.	$1V_{23}$ in a decimal fraction.	$0V_{24}$ in a decimal fraction.				
						0	0	0	0	0	0	0	0	0	0	0	0	0					0	0	0	
						1	2	n											B_1	B_2	B_3	B_7				
1	$\times$	$1V_2 \times 1V_3$	$1V_4, 1V_5, 1V_6$	$\left\{ \begin{array}{l} 1V_2 = 1V_2 \\ 1V_3 = 1V_3 \\ 1V_4 = 1V_4 \\ 1V_5 = 1V_5 \\ 1V_6 = 1V_6 \end{array} \right.$	$= 2n$	...	2	n	$2n$	$2n$	$2n$															
2	$-$	$1V_4 - 1V_1$	$2V_4$	$\left\{ \begin{array}{l} 1V_4 = 1V_4 \\ 1V_1 = 1V_1 \end{array} \right.$	$= 2n-1$	1	...	...	$2n-1$																	
3	$+$	$1V_5 + 1V_1$	$2V_5$	$\left\{ \begin{array}{l} 1V_5 = 1V_5 \\ 1V_1 = 1V_1 \end{array} \right.$	$= 2n+1$	1	...	...	...	$2n+1$																
4	$+$	$2V_5 + 2V_4$	$1V_{11}$	$\left\{ \begin{array}{l} 2V_5 = 0V_4 \\ 2V_4 = 0V_5 \end{array} \right.$	$= \frac{2n-1}{2}$	...	...	...	0	0	...	...	...	...	...	$\frac{2n-1}{2}$	$\frac{2n+1}{2}$									
5	$+$	$1V_{11} + 1V_2$	$2V_{11}$	$\left\{ \begin{array}{l} 1V_{11} = 1V_2 \\ 1V_2 = 1V_2 \end{array} \right.$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1}$	...	2	...	...	...	...	...	...	...	...	$\frac{1}{2} \cdot \frac{2n-1}{2n+1}$										
6	$-$	$0V_{13} - 2V_{11}$	$1V_{13}$	$\left\{ \begin{array}{l} 0V_{13} = 1V_{13} \\ 2V_{11} = 0V_{13} \end{array} \right.$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$	...	...	...	...	...	...	...	...	...	...	0		$-\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$								
7	$-$	$1V_3 - 1V_1$	$1V_{10}$	$\left\{ \begin{array}{l} 1V_3 = 1V_3 \\ 1V_1 = 1V_1 \end{array} \right.$	$= n-1 (=3)$	1	...	n	...	...	...	...	...	...	n-1											
8	$+$	$1V_2 + 0V_7$	$1V_7$	$\left\{ \begin{array}{l} 1V_2 = 1V_2 \\ 0V_7 = 1V_7 \end{array} \right.$	$= 2+0=2$	...	2	...	...	...	...	2														
9	$+$	$1V_6 + 1V_7$	$2V_{11}$	$\left\{ \begin{array}{l} 1V_6 = 1V_6 \\ 1V_7 = 1V_7 \end{array} \right.$	$= \frac{2n}{2} = A_1$	...	...	...	...	...	$2n$	2	...	...	...	$\frac{2n}{2} = A_1$										
10	$\times$	$1V_{21} \times 2V_{11}$	$1V_{12}$	$\left\{ \begin{array}{l} 1V_{21} = 1V_{21} \\ 2V_{11} = 1V_{12} \end{array} \right.$	$= B_1 \cdot \frac{2n}{2} = B_1 A_1$	...	...	...	...	...	...	...	...	...	...	$\frac{2n}{2} = A_1$	$B_1 \cdot \frac{2n}{2} = B_1 A_1$			B_1						
11	$+$	$1V_{12} + 1V_{13}$	$2V_{13}$	$\left\{ \begin{array}{l} 1V_{12} = 0V_{12} \\ 1V_{13} = 1V_{13} \end{array} \right.$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2}$	...	...	...	...	...	...	...	...	...	...	...	0	$\left\{ -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2} \right\}$								
12	$-$	$1V_{10} - 1V_1$	$2V_{10}$	$\left\{ \begin{array}{l} 1V_{10} = 2V_{10} \\ 1V_1 = 1V_1 \end{array} \right.$	$= n-2 (=2)$	1	...	...	...	...	...	...	...	...	n-2											
13	$-$	$1V_6 - 1V_1$	$2V_6$	$\left\{ \begin{array}{l} 1V_6 = 1V_6 \\ 1V_1 = 1V_1 \end{array} \right.$	$= 2n-1$	1	...	...	...	...	$2n-1$															
14	$+$	$1V_1 + 1V_7$	$2V_7$	$\left\{ \begin{array}{l} 1V_1 = 1V_1 \\ 1V_7 = 1V_7 \end{array} \right.$	$= 2+1=3$	1	...	...	...	...	3															
15	$+$	$2V_6 + 2V_7$	$1V_8$	$\left\{ \begin{array}{l} 2V_6 = 2V_6 \\ 2V_7 = 2V_7 \end{array} \right.$	$= \frac{2n-1}{3}$	...	...	...	...	...	$2n-1$	3	$\frac{2n-1}{3}$													
16	$\times$	$1V_8 \times 2V_{11}$	$4V_{11}$	$\left\{ \begin{array}{l} 1V_8 = 0V_8 \\ 2V_{11} = 4V_{11} \end{array} \right.$	$= \frac{2n}{2} \cdot \frac{2n-1}{3}$	...	...	...	...	...	...	...	...			$\frac{2n}{2} \cdot \frac{2n-1}{3}$										
17	$-$	$2V_6 - 1V_1$	$2V_6$	$\left\{ \begin{array}{l} 2V_6 = 2V_6 \\ 1V_1 = 1V_1 \end{array} \right.$	$= 2n-2$	1	...	...	...	...	$2n-2$															
18	$+$	$1V_1 + 2V_7$	$2V_7$	$\left\{ \begin{array}{l} 1V_1 = 1V_1 \\ 2V_7 = 2V_7 \end{array} \right.$	$= 3+1=4$	1	...	...	...	...	...	4														
19	$+$	$2V_6 + 2V_7$	$1V_9$	$\left\{ \begin{array}{l} 2V_6 = 2V_6 \\ 2V_7 = 2V_7 \end{array} \right.$	$= 2n-2$	...	...	...	...	...	$2n-2$	4	$\frac{2n-2}{4}$	...		$\left\{ \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} \right\}$										
20	$\times$	$1V_9 \times 4V_{11}$	$2V_{11}$	$\left\{ \begin{array}{l} 1V_9 = 0V_9 \\ 4V_{11} = 2V_{11} \end{array} \right.$	$= \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} = A_3$	...	...	...	...	...	...	...	...		0											
21	$\times$	$1V_{22} \times 2V_{11}$	$0V_{12}$	$\left\{ \begin{array}{l} 1V_{22} = 1V_{22} \\ 2V_{11} = 0V_{12} \end{array} \right.$	$= B_3 \cdot \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} = B_3 A_3$	...	...	...	...	...	...	...	...		...	0	$B_3 A_3$			B_3						
22	$+$	$2V_{12} + 2V_{13}$	$2V_{13}$	$\left\{ \begin{array}{l} 2V_{12} = 0V_{12} \\ 2V_{13} = 1V_{13} \end{array} \right.$	$= A_0 + B_1 A_1 + B_3 A_3$	...	...	...	...	...	...	...	...		...	...	0	$\{ A_3 + B_1 A_1 + B_3 A_3 \}$								
23	$-$	$2V_{10} - 1V_1$	$2V_{10}$	$\left\{ \begin{array}{l} 2V_{10} = 2V_{10} \\ 1V_1 = 1V_1 \end{array} \right.$	$= n-3 (=1)$	1	...	...	...	...	...	...	...		n-3											
Here follows a repetition of Operations thirteen to twenty-three.																										
24	$+$	$4V_{13} + 0V_{24}$	$1V_{24}$	$\left\{ \begin{array}{l} 4V_{13} = 0V_{13} \\ 0V_{24} = 1V_{24} \end{array} \right.$	$= B_7$	...	...	...	...	...	...	...	...	...	...	...	...	...					B_7			
25	$+$	$1V_1 + 1V_3$	$1V_3$	$\left\{ \begin{array}{l} 1V_1 = 1V_1 \\ 1V_3 = 1V_3 \end{array} \right.$	$= n+1 = 4+1=5$	1	...	n+1	...	...	0	0														
by a Variable-card. by a Variable card.																										



1947

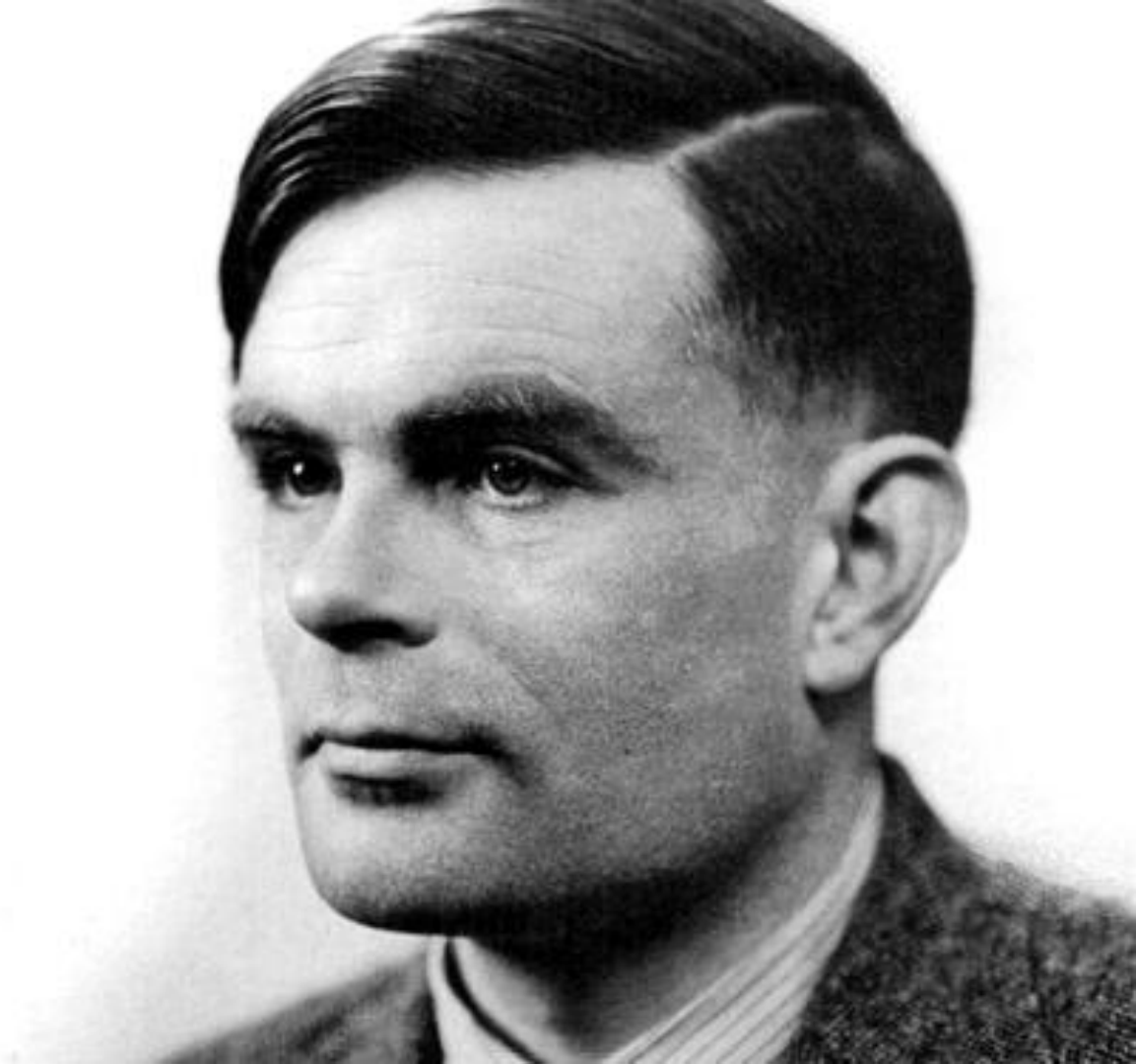
Manual Program Analysis

Alan Turing.

"Checking a large routine."

EDSAC Inaugural Conference,
1949.

1949



Program Verification

Given a program and a specification, show that the program conforms to the specification by creating a formal proof.

Specification: The program should not crash.

```
int main()
{
    unsigned int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    if (a + b == c && c - b != a)
        crash();
}
```



VERIFIED

Specification: The program should not crash.

```
int main()
{
    unsigned int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    if (a+b+c != 0 && pow(a, 3) + pow(b, 3) == pow(c, 3))
        crash();
}
```



Program Testing

Given a program and a specification, show that the program *does not* conform to the specification by providing a *counterexample*.

Specification: The program should not crash.

```
int main()
{
    unsigned int a, b, c;
    scanf("%d %d %d", &a, &b, &c);

    if (a+b+c != 0 && pow(a, 2) + pow(b, 2) == pow(c, 2))
        crash();
}
```

Counterexample:

a == 3
b == 4
c == 5

1950s

<http://secretsofconsulting.blogspot.com/2017/02/fuzz-testing-and-fuzz-history.html>

Specification

What should hold about the program?

Logical Properties
Absence of Crashes

Goal

What do we want to achieve regarding the specification?

Verification
Testing

Technique

How will we achieve the goal?

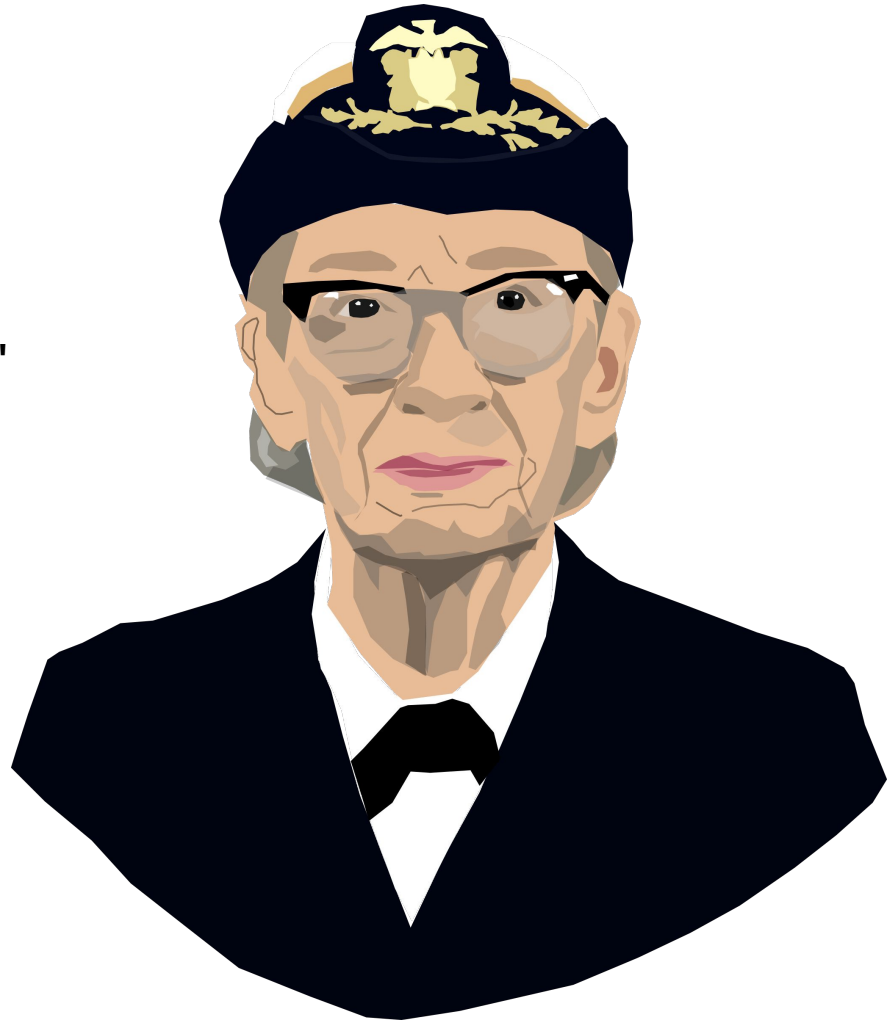
Manual

Need for Automated Techniques

Grace Hopper.

"The Education of a Computer."

Proceedings of the 1952 ACM
national meeting, 1952.



1952

Specification

What should hold about the program?

Logical Properties
Absence of Crashes
Type Safety
Efficiency

Goal

What do we want to achieve regarding the specification?

Verification
Testing
Transformation

Technique

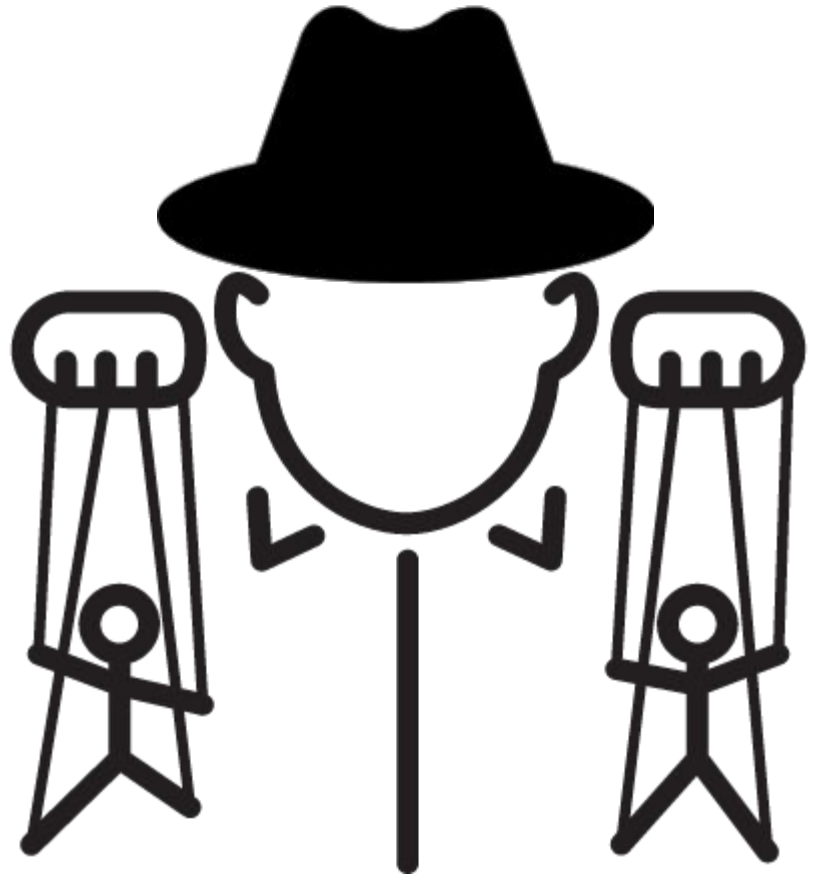
How will we achieve the goal?

Manual

Robert Graham.

"Protection in an information
processing utility."

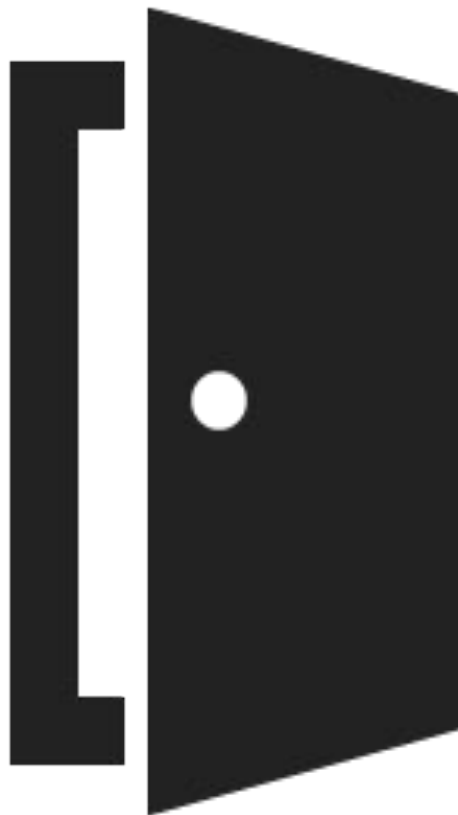
Communications of the ACM,
1968.



1968

Ken Thompson.

"Reflections on trusting
trust." Communications of the
ACM, 1984.



1984

Specification

What should hold about the program?

Goal

What do we want to achieve regarding the specification?

Technique

How will we achieve the goal?

Logical Properties
Absence of Crashes
Type Safety
Efficiency
Memory Safety
Information Disclosure
Authentication

Verification
Testing
Transformation

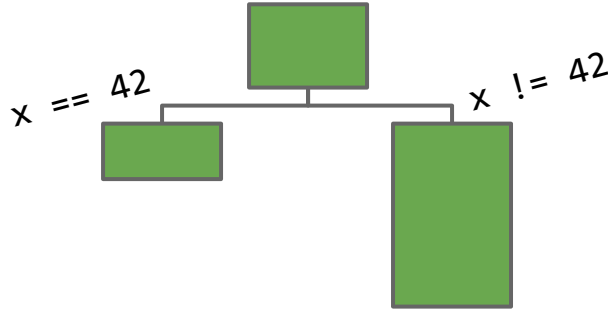
Manual

Automated Techniques

Prerequisites

Basic Block

Constraints



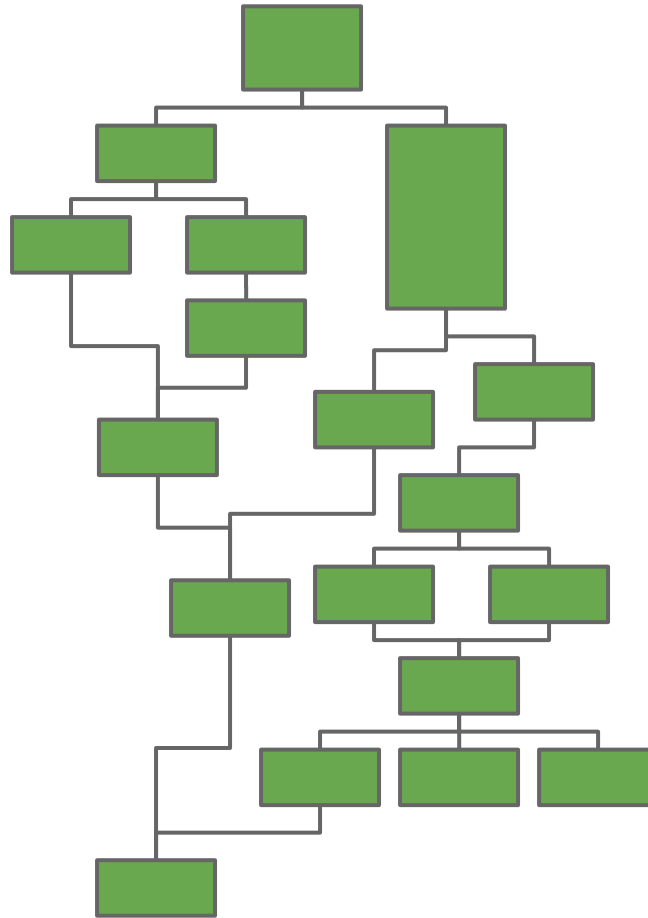
```
x = input()
if x == 42:
    print "Correct"
else:
    print "No"
```

Prerequisites

Basic Block

Constraints

Control Flow Graph



```
x = input()
if x == 42:
    print "Correct"
else:
    print "No"
    if x == 1337:
        print "Fine"
```

```
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
```

Prerequisites

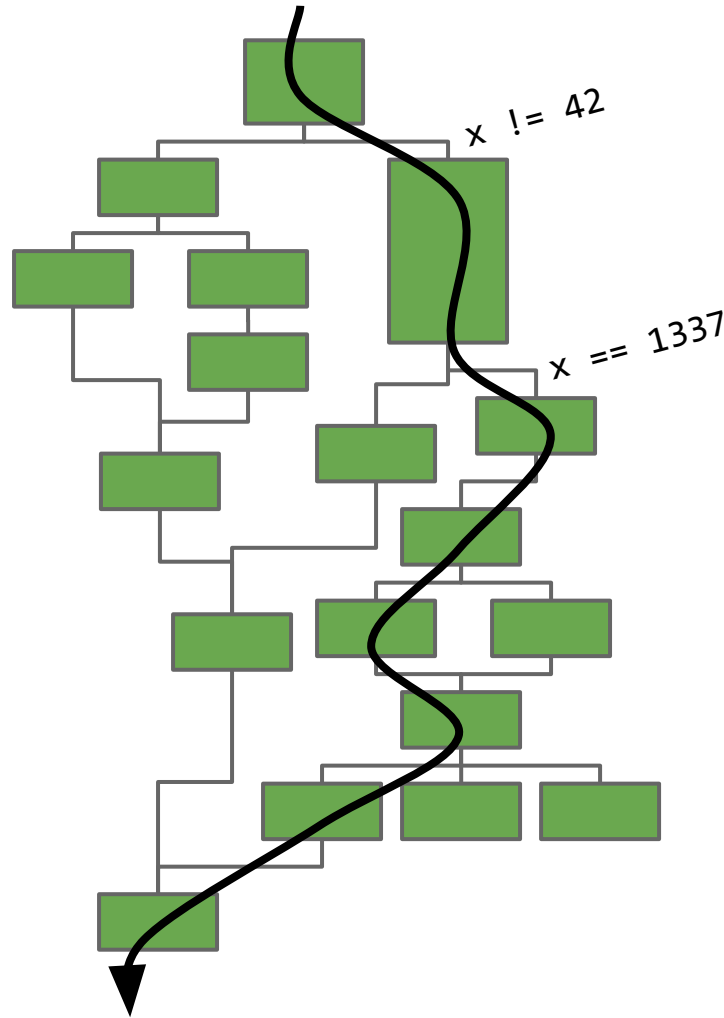
Basic Block

Constraints

Control Flow Graph

Path

Path Predicates



```
x = input()
if x == 42:
    print "Correct"
else:
    print "No"
    if x == 1337:
        print "Fine"
```

.....
.....
.....
.....
.....
.....
.....
.....
.....

The Rise of Symbolic Execution

Robert Boyer, et al.

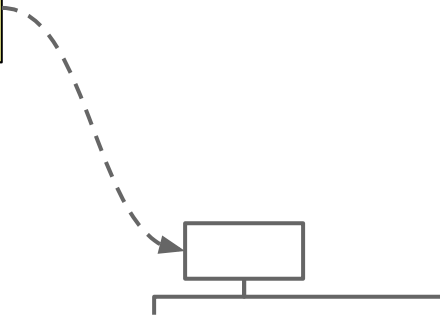
"SELECT—a formal system for testing and debugging programs by symbolic execution."

ACM SigPlan Notices, 1975.

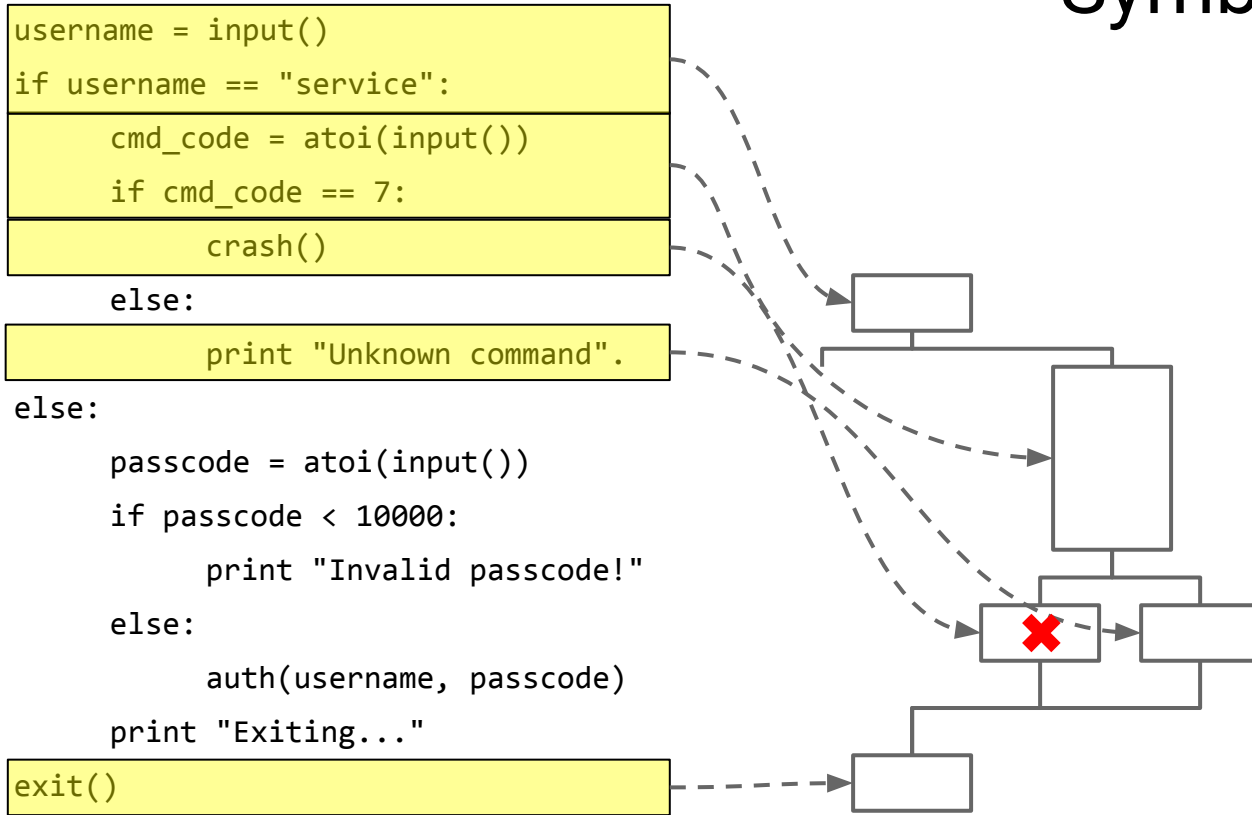
1975

Symbolic Execution

```
username = input()
if username == "service":
    cmd_code = atoi(input())
    if cmd_code == 7:
        crash()
    else:
        print "Unknown command".
else:
    passcode = atoi(input())
    if passcode < 10000:
        print "Invalid passcode!"
    else:
        auth(username, passcode)
    print "Exiting..."
exit()
```

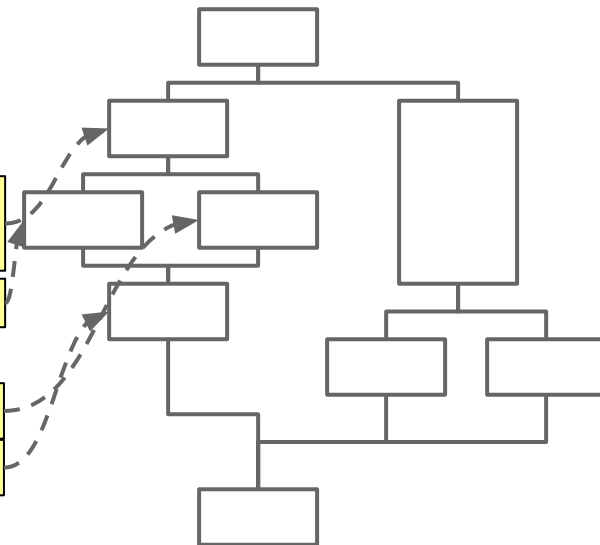


Symbolic Execution



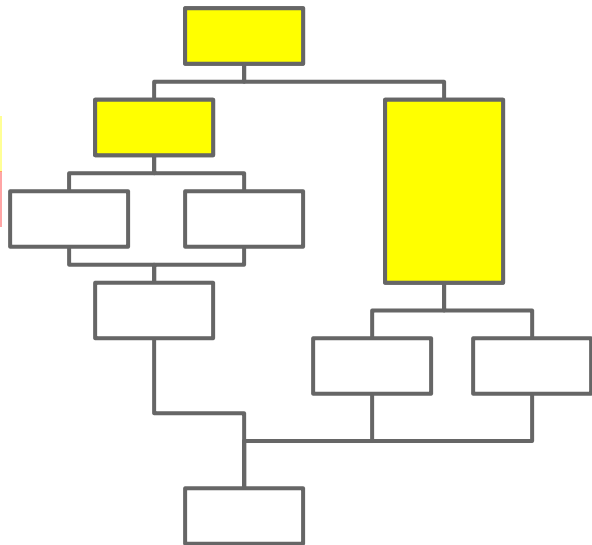
Symbolic Execution

```
username = input()
if username == "service":
    cmd_code = atoi(input())
    if cmd_code == 7:
        crash()
    else:
        print "Unknown command".
else:
    passcode = atoi(input())
    if passcode < 10000:
        print "Invalid passcode!"
    else:
        auth(username, passcode)
        print "Exiting..."
exit()
```



Symbolic Execution

```
username = input()
if username == "service":
    cmd_code = atoi(input())
    if cmd_code == 7:
        crash()
    else:
        print "Unknown command".
else:
    passcode = atoi(input())
    if passcode < 10000:
        print "Invalid passcode!"
    else:
        auth(username, passcode)
    print "Exiting..."
exit()
```



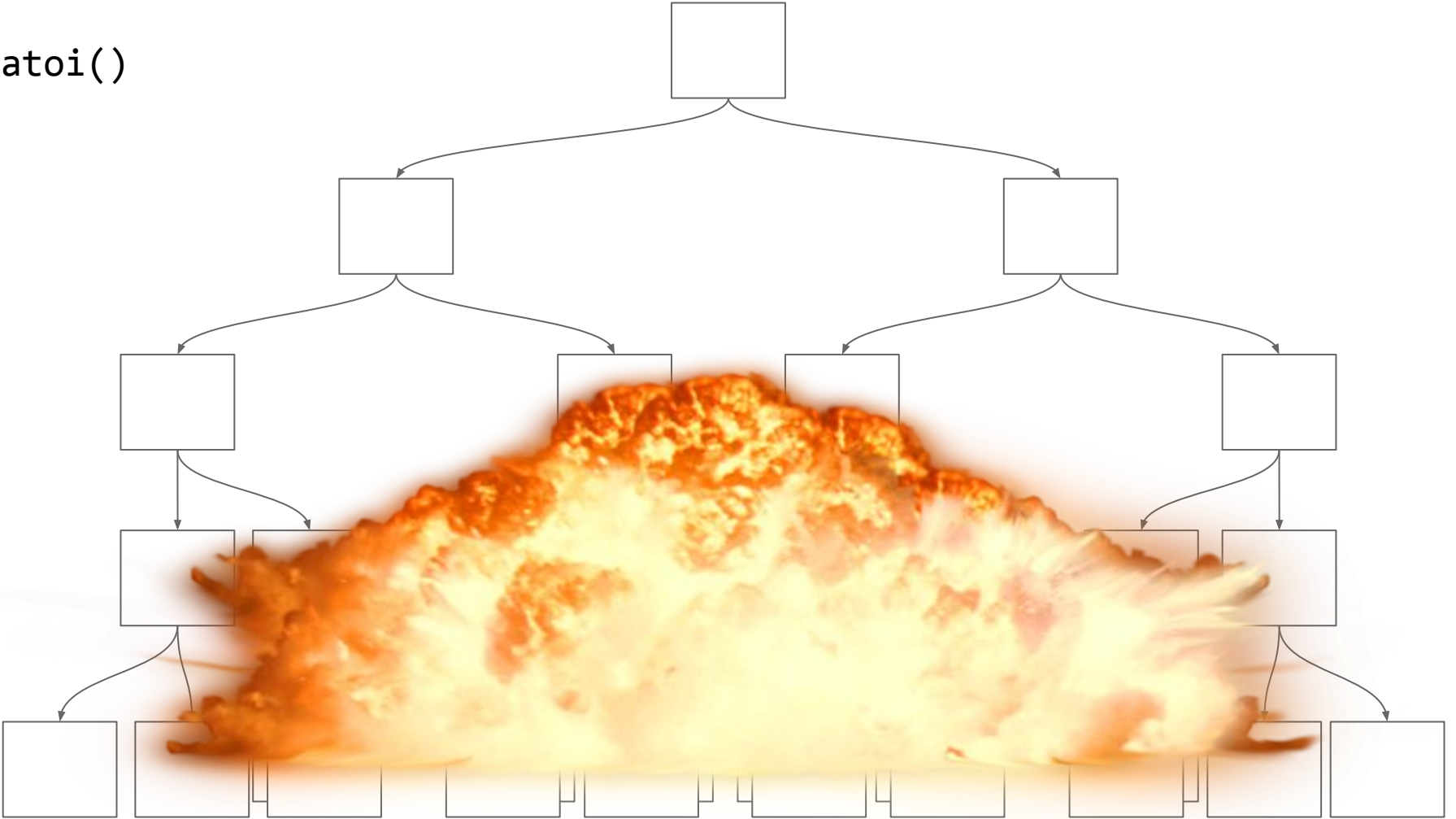
Constraints

username = ???

username
==
"service"

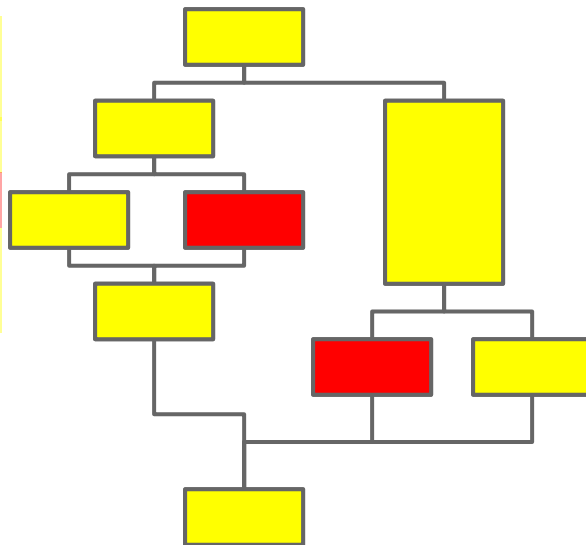
username
!=
"service"

atoi()



Symbolic Execution

```
username = input()
if username == "service":
    cmd_code = atoi(input())
    if cmd_code == 7:
        crash()
    else:
        print "Unknown command".
else:
    passcode = atoi(input())
    if passcode < 10000:
        print "Invalid passcode!"
    else:
        auth(username, passcode)
    print "Exiting..."
exit()
```



Constraints

username = ???

```
username
==
"service"
```

```
username
!=
"service"
```

```
cmd_code
!=
"7"
```

```
passcode
<=
1000
```

Number of Operation.	Nature of Operation.	Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data.										Working Variables.					Result Variables.			
						$1V_1$ 0 0 0	$1V_2$ 0 0 2	$1V_3$ 0 0 4	$0V_4$ 0 0 0	$0V_5$ 0 0 0	$0V_6$ 0 0 0	$0V_7$ 0 0 0	$0V_8$ 0 0 0	$0V_9$ 0 0 0	$0V_{10}$ 0 0 0	$0V_{11}$ 0 0 0	$0V_{12}$ 0 0 0	$0V_{13}$ 0 0 0	$1V_{21}$ 0 0 0	$1V_{22}$ 0 0 0	$1V_{23}$ 0 0 0	$0V_{24}$ 0 0 0		
						1	2	n										B_1 in a decimal fraction.	B_2 in a decimal fraction.	B_3 in a decimal fraction.	B_4 in a decimal fraction.			
1	$\times$	$1V_2 \times 1V_3$	$1V_4, 1V_5, 1V_6$	$1V_2 = 1V_2$ $1V_3 = 1V_3$ $1V_4 = 1V_4$ $1V_5 = 1V_5$ $1V_6 = 1V_6$	$= 2n$	...	n	2n	2n	2n														
2	$-$	$1V_4 - 1V_1$	$2V_4$	$1V_4 = 1V_4$ $1V_5 = 1V_5$ $1V_6 = 1V_6$	$= 2n - 1$	1	...	2n - 1																
3	$+$	$1V_5 + 1V_1$	$2V_5$	$1V_5 = 1V_5$ $1V_6 = 1V_6$	$= 2n + 1$	1	...	2n + 1																
4	$+$	$2V_5 + 2V_4$	$1V_{11}$	$1V_{11} = 1V_{11}$ $1V_{12} = 1V_{12}$ $1V_{13} = 1V_{13}$	$= \frac{2n-1}{2n+1}$	...	...	0	0							$\frac{2n-1}{2n+1}$								
5	$+$	$1V_{11} + 1V_2$	$2V_{11}$	$1V_{11} = 1V_{11}$ $1V_{12} = 1V_{12}$ $1V_{13} = 1V_{13}$	$= \frac{1}{2} \cdot \frac{2n-1}{2n+1}$	2	...									$\frac{1}{2} \cdot \frac{2n-1}{2n+1}$								
6	$-$	$0V_{13} - 2V_{11}$	$1V_{13}$	$1V_{13} = 1V_{13}$ $1V_{14} = 1V_{14}$ $1V_{15} = 1V_{15}$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} = A_0$	...	...									0								
7	$-$	$1V_3 - 1V_1$	$1V_{10}$	$1V_3 = 1V_3$ $1V_4 = 1V_4$ $1V_5 = 1V_5$	$= n - 1 (= 3)$	1	...								n - 1									
8	$+$	$1V_2 + 0V_7$	$1V_7$	$1V_2 = 1V_2$ $0V_7 = 0V_7$ $1V_6 = 1V_6$	$= 2 + 0 = 2$	2	...				2													
9	$+$	$1V_6 + 1V_7$	$2V_{11}$	$1V_6 = 1V_6$ $1V_7 = 1V_7$ $0V_{11} = 0V_{11}$	$= \frac{2n}{2} = A_1$	...	...			2n	2					$\frac{2n}{2} = A_1$								
10	$\times$	$1V_{21} \times 1V_{11}$	$1V_{12}$	$1V_{21} = 1V_{21}$ $1V_{11} = 1V_{11}$ $1V_{12} = 1V_{12}$	$= B_1 \cdot \frac{2n}{2} = B_1 A_1$	...	...									$\frac{2n}{2} = A_1$	$B_1 \cdot \frac{2n}{2} = B_1 A_1$				B_1			
11	$+$	$1V_{12} + 1V_{13}$	$2V_{13}$	$1V_{12} = 1V_{12}$ $1V_{13} = 1V_{13}$ $1V_{14} = 1V_{14}$	$= -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2}$	...	...										0				$\left\{ -\frac{1}{2} \cdot \frac{2n-1}{2n+1} + B_1 \cdot \frac{2n}{2} \right\}$			
12	$-$	$1V_{10} - 1V_1$	$2V_{10}$	$1V_{10} = 1V_{10}$ $1V_1 = 1V_1$ $1V_2 = 1V_2$	$= n - 2 (= 2)$	1	...								n - 2									
13	$-$	$1V_6 - 1V_1$	$2V_6$	$1V_6 = 1V_6$ $1V_7 = 1V_7$ $1V_8 = 1V_8$	$= 2n - 1$	1	...			2n - 1														
14	$+$	$1V_1 + 1V_7$	$2V_7$	$1V_1 = 1V_1$ $1V_7 = 1V_7$ $1V_8 = 1V_8$	$= 2 + 1 = 3$	1	...				3													
15	$+$	$2V_6 + 2V_7$	$1V_8$	$1V_6 = 1V_6$ $1V_7 = 1V_7$ $1V_8 = 1V_8$	$= \frac{2n-1}{3}$	...	...			2n - 1	3	$\frac{2n-1}{3}$												
16	$\times$	$1V_8 \times 2V_{11}$	$4V_{11}$	$1V_8 = 1V_8$ $1V_{11} = 1V_{11}$ $1V_{12} = 1V_{12}$	$= \frac{2n}{2} \cdot \frac{2n-1}{3}$	...	...					0				$\frac{2n}{2} \cdot \frac{2n-1}{3}$								
17	$-$	$2V_6 - 1V_1$	$2V_6$	$1V_6 = 1V_6$ $1V_7 = 1V_7$ $1V_8 = 1V_8$	$= 2n - 2$	1	...			2n - 2														
18	$+$	$1V_1 + 2V_7$	$2V_7$	$1V_1 = 1V_1$ $1V_7 = 1V_7$ $1V_8 = 1V_8$	$= 3 + 1 = 4$	1	...				4													
19	$+$	$2V_6 - 2V_7$	$1V_9$	$1V_6 = 1V_6$ $1V_7 = 1V_7$ $1V_8 = 1V_8$	$= \frac{2n-2}{4}$	...	...			2n - 2	4	$\frac{2n-2}{4}$				$\left\{ \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} \right\}$								
20	$\times$	$1V_9 \times 1V_{11}$	$4V_{11}$	$1V_9 = 1V_9$ $1V_{11} = 1V_{11}$ $1V_{12} = 1V_{12}$	$= \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} = A_3$	...	...					0												
21	$\times$	$1V_{22} \times 1V_{11}$	$0V_{12}$	$1V_{22} = 1V_{22}$ $1V_{11} = 1V_{11}$ $1V_{12} = 1V_{12}$	$= B_3 \cdot \frac{2n}{2} \cdot \frac{2n-1}{3} \cdot \frac{2n-2}{4} = B_3 A_3$	...	...									0	$B_3 A_3$				B_3			
22	$+$	$2V_{12} + 2V_{13}$	$2V_{13}$	$1V_{12} = 1V_{12}$ $1V_{13} = 1V_{13}$ $1V_{14} = 1V_{14}$	$= A_0 + B_1 A_1 + B_3 A_3$	...	...										0				$\{ A_3 + B_1 A_1 + B_3 A_3 \}$			
23	$-$	$2V_{10} - 1V_1$	$2V_{10}$	$1V_{10} = 1V_{10}$ $1V_1 = 1V_1$ $1V_2 = 1V_2$	$= -3 (= 1)$	1	...								n - 3									
Here follows a repetition of Operations thirteen to twenty-three.																								
24	$+$	$1V_{13} + 0V_{24}$	$1V_{24}$	$1V_{13} = 1V_{13}$ $0V_{24} = 0V_{24}$ $1V_{25} = 1V_{25}$	$= B_7$	...	...																	
25	$+$	$1V_1 + 1V_3$	$1V_3$	$1V_1 = 1V_1$ $1V_3 = 1V_3$ $0V_4 = 0V_4$	$= n + 1 = 4 + 1 = 5$	1	...	n + 1			0	0									B_7			

Here follows a repetition of Operations thirteen to twenty-three.

Variables acted upon.	Variables receiving results.	Indication of change in the value on any Variable.	Statement of Results.	Data					
				1V_1 ○ 0 0 1 1	1V_2 ○ 0 0 2 2	1V_3 ○ 0 0 4 n	0V_4 ○ 0 0 0 	0V_5 ○ 0 0 0 	0V_6 ○ 0 0 0
$V_2 \times {}^1V_3$	${}^1V_4, {}^1V_5, {}^1V_6$	$\begin{cases} {}^1V_2 = {}^1V_2 \\ {}^1V_3 = {}^1V_3 \end{cases}$	$= 2n$	...	2	n	$2n$	$2n$	$2n$
$V_4 - {}^1V_1$	2V_4	$\begin{cases} {}^1V_4 = {}^2V_4 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 2n - 1$	1	...	...	$2n - 1$		
$V_5 + {}^1V_1$	2V_5	$\begin{cases} {}^1V_5 = {}^2V_5 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= 2n + 1$	1	...	...	...	$2n + 1$	
$V_6 \div {}^2V_4$	${}^1V_{11}$	$\begin{cases} {}^2V_5 = {}^0V_5 \\ {}^2V_4 = {}^0V_4 \end{cases}$	$= \frac{2n - 1}{2n + 1}$	...	...	...	0	0	...
$V_{11} \div {}^1V_2$	${}^2V_{11}$	$\begin{cases} {}^1V_{11} = {}^2V_{11} \\ {}^1V_2 = {}^1V_2 \end{cases}$	$= \frac{1}{2} \cdot \frac{2n - 1}{2n + 1}$	...	2	...	...	...	...
$V_{13} - {}^2V_{11}$	${}^1V_{13}$	$\begin{cases} {}^2V_{11} = {}^0V_{11} \\ {}^0V_{13} = {}^1V_{13} \end{cases}$	$= -\frac{1}{2} \cdot \frac{2n - 1}{2n + 1} = A_0$	...	...	...	...	...	...
$V_3 - {}^1V_1$	${}^1V_{10}$	$\begin{cases} {}^1V_3 = {}^1V_3 \\ {}^1V_1 = {}^1V_1 \end{cases}$	$= n - 1 (= 3)$	1	...	n	...	...	...

Specification

What should hold about the program?

Logical Properties
Absence of Crashes
Type Safety
Efficiency
Memory Safety
Information Disclosure
Authentication

Goal

What do we want to achieve regarding the specification?

Verification
Testing
Transformation

Technique

How will we achieve the goal?

Manual
Symbolic Execution

Emergence of Static Analysis

Patrick & Radhia Cousot.

"Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints"

ACM Symposium on Principles of Programming Languages, 1977,

1977

Specification

What should hold about the program?

Logical Properties
Absence of Crashes
Type Safety
Efficiency
Memory Safety
Information Disclosure
Authentication

Goal

What do we want to achieve regarding the specification?

Verification
Testing
Transformation

Technique

How will we achieve the goal?

Manual
Symbolic Execution
Abstract Interpretation

Fuzzing Appears

Joe W. Duran, et al.

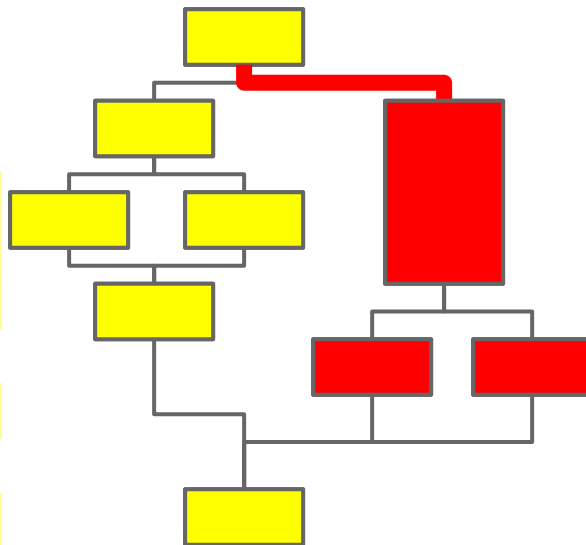
"A report on random testing".

ACM SIGSOFT International Conference on Software Engineering,
1981.

1981

Fuzzing

```
username = input()
if username == "service":
    cmd_code = atoi(input())
    if cmd_code == 7:
        crash()
    else:
        print "Unknown command".
else:
    passcode = atoi(input())
    if passcode < 10000:
        print "Invalid passcode!"
    else:
        auth(username, passcode)
        print "Exiting..."
exit()
```



Test Cases

"asdf:111"

"asDA:111"

"asDAAA:1111"

"asDALA:11121"

"axDOO:15129"

"asFOO:75129"

IBM CALCULATOR INSTRUCTION CARD

IDENTIFICATION

LOCATION

OPERATION

ADDRESS

SYMBOLIC

LOCATION

1. $\frac{1}{2}$

PART _____

THE

10

PREFACE

COMMENT

99999999

99

9

9

99

99

●

910

94

91

3:9

99

99

99

9

19

999999

1 2 3 4 5 6 7 8 9
1011 021162

10 11 12 13 14 15 16 17 18

19 20 21 22 23 24 25 26 27

28 29 30 31 32 33 34 35 36

27 28 29 30 31 32 33 34 35

46 47 48 49 50 51 52 53 54

56 57 58 59 60 61 62 63

4 65 66 67 68 69 70 71 72

1 74 75 76 77 78 79 80

MM 02462

Specification

What should hold about the program?

Logical Properties
Absence of Crashes
Type Safety
Efficiency
Memory Safety
Information Disclosure
Authentication

Goal

What do we want to achieve regarding the specification?

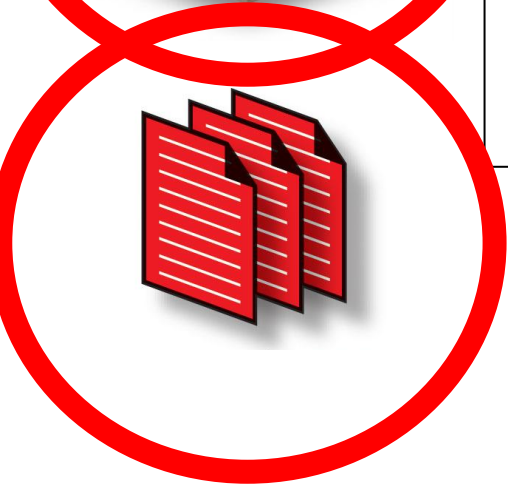
Verification
Testing
Transformation

Technique

How will we achieve the goal?

Manual
Symbolic Execution
Abstract Interpretation
Fuzzing

The Program Analysis Nursery

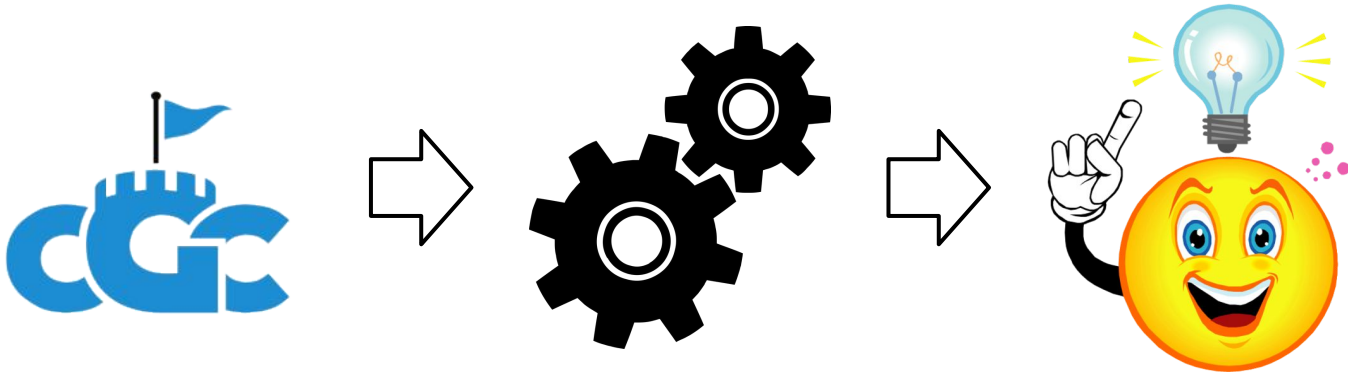




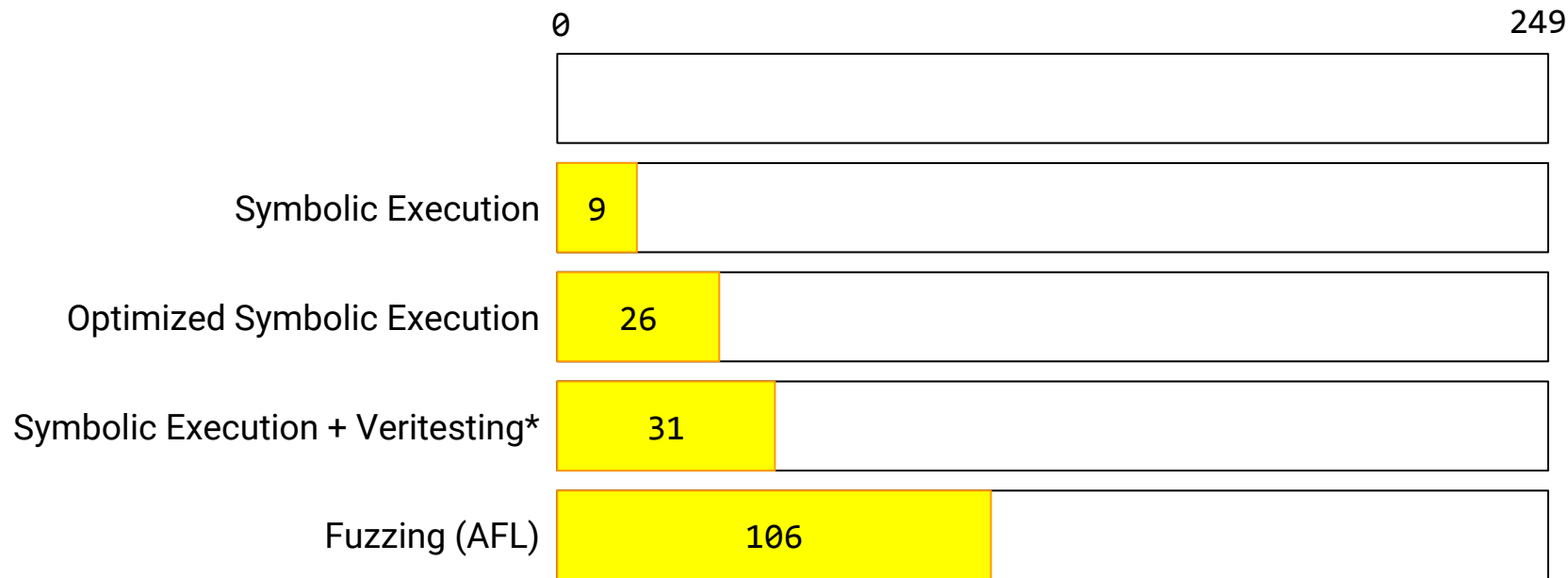
CYBER
GRAND_CHALLENGE

The Program Analysis Nursery

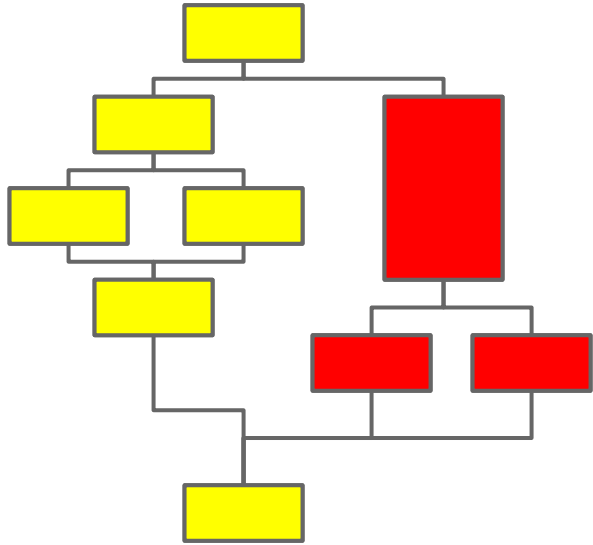
- 249 programs
- Source code available
- Range of vulnerability classes
- Documented vulnerabilities
- Simple OS model
- Explicit security specifications!



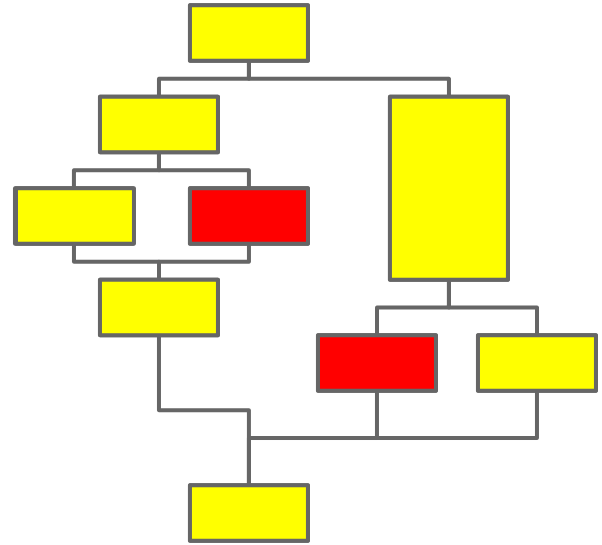
Nursery Experiments



Fuzzing

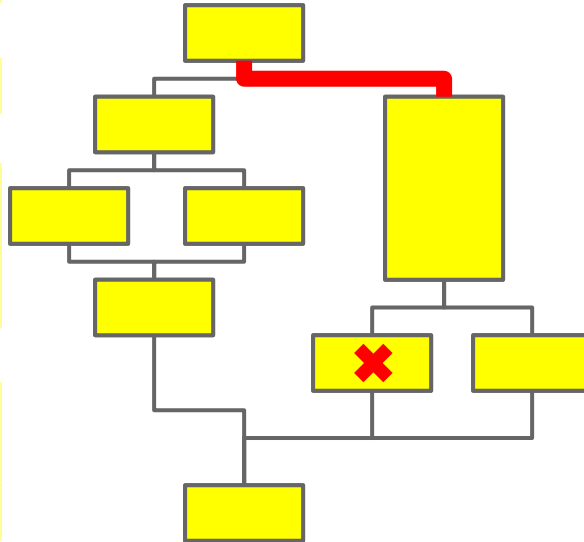


Symbolic Execution



Drilling

```
username = input()
if username == "service":
    cmd_code = atoi(input())
    if cmd_code == 7:
        crash()
else:
    print "Unknown command".
else:
    passcode = atoi(input())
    if passcode < 10000:
        print "Invalid passcode!"
    else:
        auth(username, passcode)
        print "Exiting..."
exit()
```



Test Cases

"asdf:111"

"asDAAA:1111"

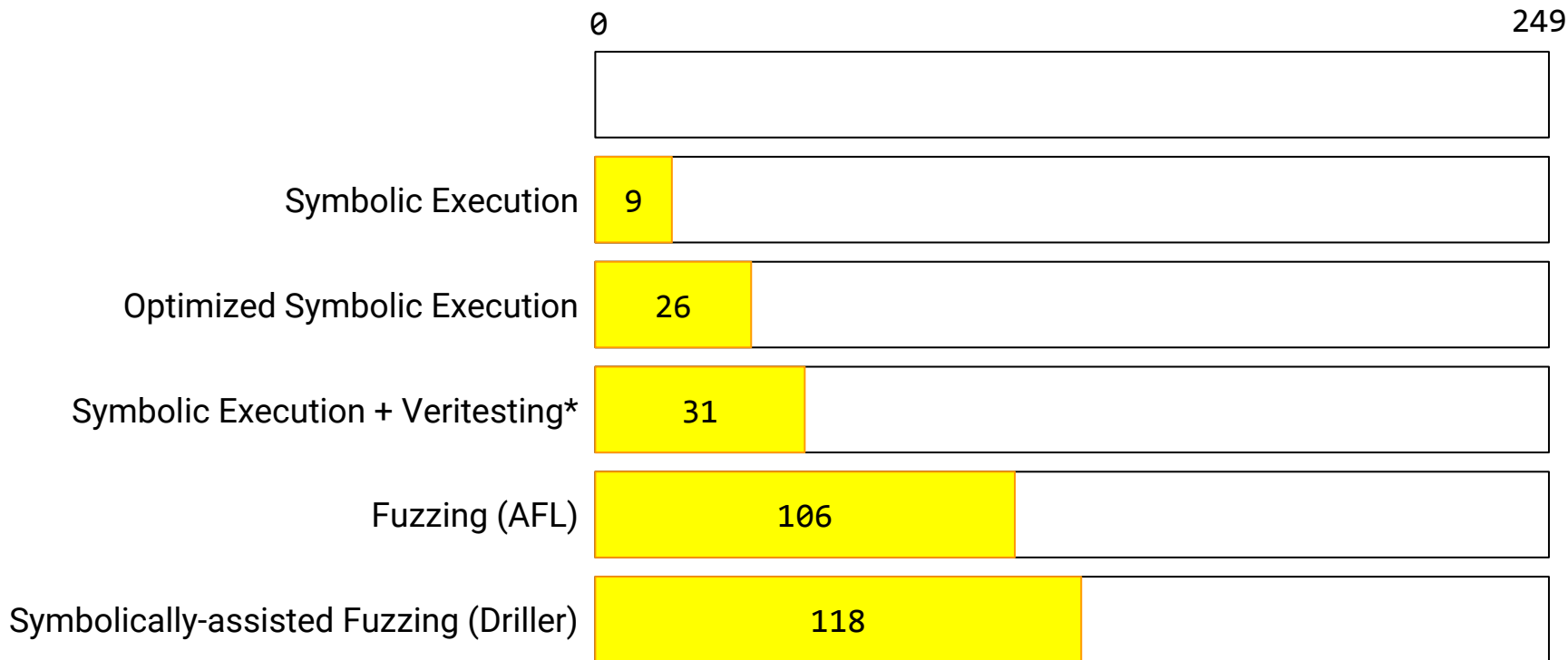
username == "service"
cmd_code != "7"

"service:5"

"servic3:5"

"service:7"

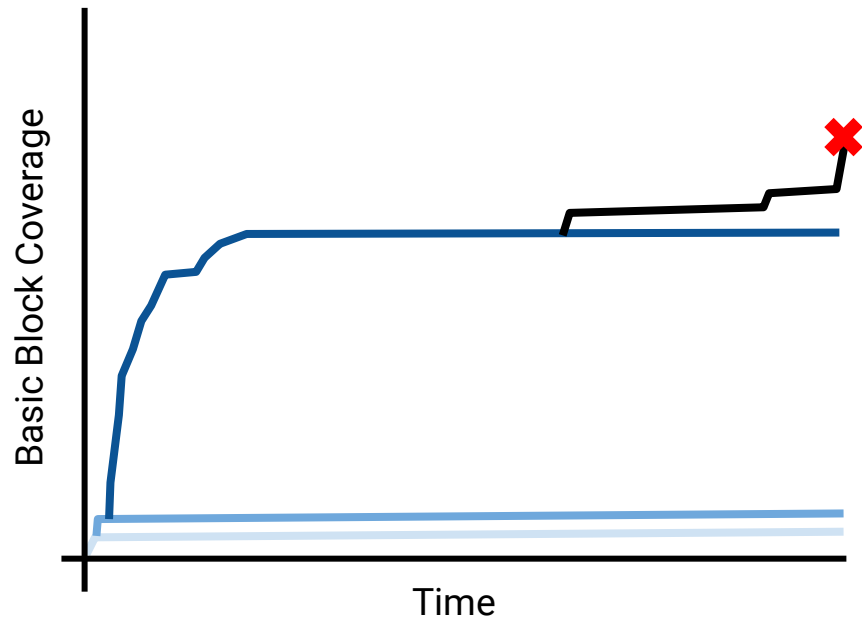
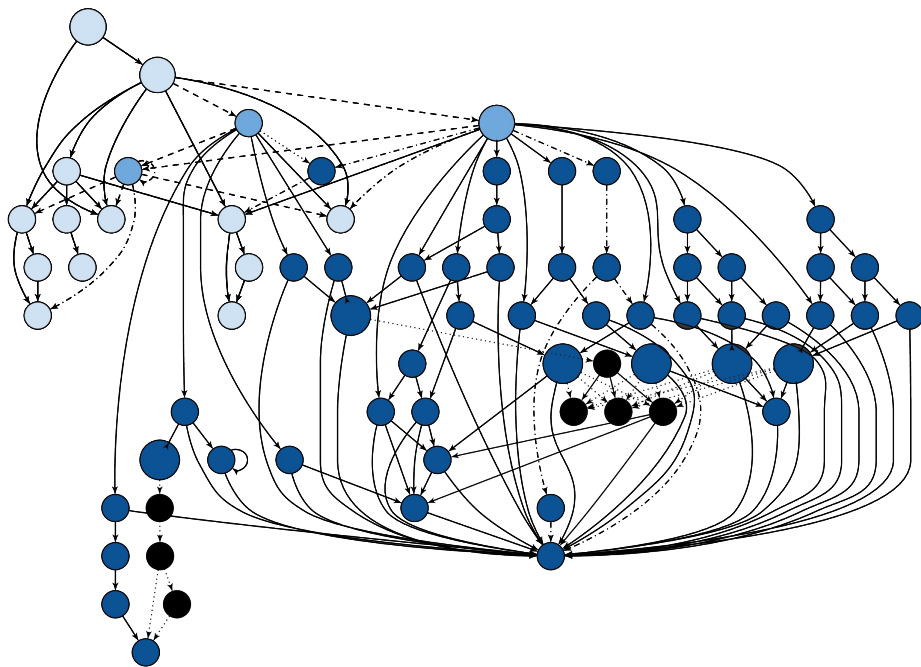
Nursery Experiments



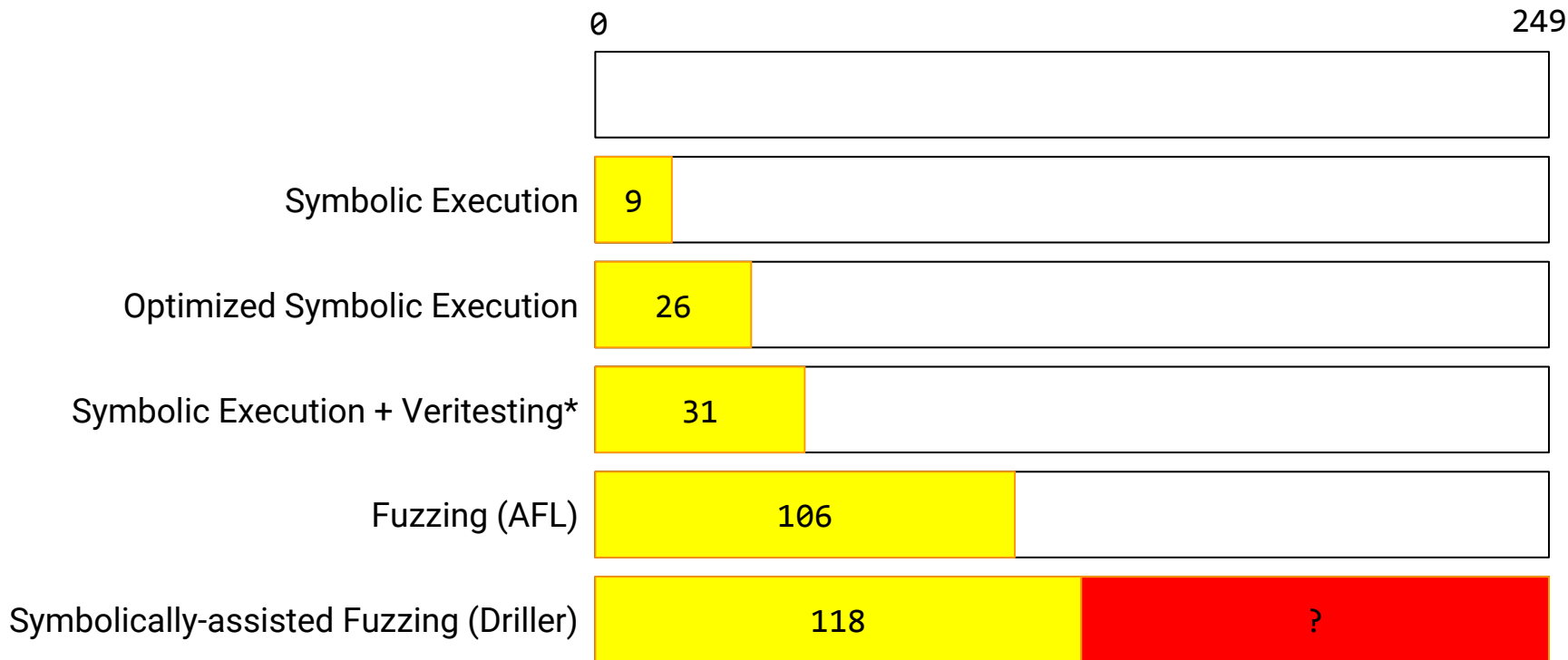
Driller Results

Applicability varies by program.

Where it was needed, Driller increased block coverage by an average of 71%.



Nursery Experiments



Join in!

Contribute to open-source frameworks!



<http://angr.io>

Come do research!

I am actively looking for students, interns, etc!

Yan Shoshitaishvili

Me: @Zardus yans@yancomm.net

Team: @Shellphish team@shellphish.net

This presentation: <https://goo.gl/57BAoc>

Questions?

